重大 AI 更新提醒

10-28 | llama.cpp 同日连发三个版本：v0.8.0 大版本扩展多模态与实时音频，CUDA 侧连续优化 MoE 推理

本期导读

今天 llama.cpp 项目在数小时内连续发布三个版本，构成一次值得关注的集中更新。v0.8.0 是功能面最广的大版本：新增 LiquidAI LFM3 全模态家族、实验性流式音频 API、 v1/realtime 与 tts 端点，并落地 DeepSeek V4.1 Flash 等一批新架构。两个滚动构建 b11953 与 b11962 则聚焦 CUDA 后端，分别针对 384 专家的 top-k MoE 融合和 MMQ 中 MoE gate/up 与 GLU 的算子融合做优化。对本地推理与自建服务团队而言，这是一次功能与性能同时推进的更新。

已检查来源

GitHub Release - ggml-org/llama.cpp b11962、GitHub Release - ggml-org/llama.cpp v0.8.0、GitHub Release - ggml-org/llama.cpp b11953

1. llama.cpp v0.8.0：新增 LiquidAI LFM3 全模态家族与实验性流式音频 API

来源　GitHub Release | 类型　GitHub Release | 主题　智能体与开发者平台　时间　10-28 07:47 | 分数　7

是什么

这是 ggml-org/llama.cpp 的 v0.8.0 正式版本发布。它一次性引入 LiquidAI LFM3 家族（覆盖文本、视觉、音频与 omni 四种形态），在 mtmd 中加入实验性的流式音频 API，并通过新增的　v1/realtime 与　tts 端点把实时音频输入输出接入服务端。同时合入一批新模型架构支持，包括 DeepSeek V4.1 Flash、SenseNova U1、Beam、Kolibri-1、Flint、Muse Spark、Granite Switch Decision，以及来自 LiquidAI、百度和 NVIDIA 的决策　嵌入模型。核心层把稀疏注意力（QSA/DSA）重写到融合掩码算子上，引入面向主机驻留 MoE 专家的 50/50 SLRU 缓存，并清理了 no_alloc 内存处理；服务端改进了路由内存管理与工具调用；UI 支持直接从 HF Hub 浏览下载模型、搜索与压缩对话；ggml 升级到 v0.28.0，带来 GGML_OP_TOP_K_MASK、　1 专家索引语义等变化。

通俗解释

可以把 llama.cpp 理解成一个让各种大模型能在普通电脑、手机、服务器上跑起来的　通用发动机　。这次 v0.8.0 相当于给发动机同时加装了几套新配件：第一，能识别更多型号的　燃料　，也就是更多新模型架构，比如 DeepSeek V4.1 Flash；第二，新增了　耳朵和嘴巴　，也就是流式音频接口，你可以像打电话一样把声音实时送进模型、再把合成语音实时拿回来，而不是等一整段录音处理完；第三，给 MoE 这类　按需调用专家　的模型加了一个更聪明的缓存策略，好比图书馆把最常被借的书放在门口一半、另一半放书库，减少来回搬运；第四，界面里可以直接从 Hugging Face 下载模型、搜索历史对话，省去手动找文件的步骤。

主要作用

解决的是本地与自建推理栈　模型支持滞后、多模态能力割裂、服务端接口不统一　的问题。此前要在 llama.cpp 上跑新发布的开源模型往往需要等社区适配，音频能力也缺少标准化的实时通道；v0.8.0 把新架构、全模态输入输出和实时音频端点一次性补齐，让开发者不必在多个工具之间拼接，就能用同一套二进制覆盖文本、视觉、语音场景，并直接对外提供兼容 OpenAI 风格的服务接口。

核心功能

一是 LiquidAI LFM3 全模态家族支持，文本、视觉、音频、omni 四类模型可在同一框架内加载；二是实验性流式音频 API 与　v1/realtime、　tts 端点，支持实时音频进出服务端；三是核心算子与内存优化，包括 QSA/DSA 稀疏注意力融合掩码、面向主机驻留 MoE 专家的 50/50 SLRU 缓存、更干净的 no_alloc 内存处理；四是服务端与 UI 增强，含路由内存管理、工具调用改进、从 HF Hub 直接浏览下载模型、对话搜索与压缩。

对比判断

候选摘要未给出与上一版本或其他推理框架的 benchmark、吞吐或成本对比数据，暂无可信公开对比数据。

适合谁

适合在本地或私有环境部署大模型的开发者、需要实时语音交互能力的应用团队、维护多模型推理网关的平台工程师，以及关注 MoE 与稀疏注意力实现细节的研究者。

直达链接　https://github.com/ggml-org/llama.cpp/releases/tag/v0.8.0

2. llama.cpp b11953：CUDA 后端为 384 专家 MoE 增加 top-k 融合

来源　GitHub Release | 类型　GitHub Release | 主题　推理性能与基础设施　时间　10-28 05:11 | 分数　7

是什么	这是 llama.cpp 的滚动构建 b11953，核心变更是 CUDA 后端新增针对 384 专家规模 MoE 模型的 top-k 融合实现（PR #31288，提交说明中标注 Assisted-by: Codex）。它属于推理内核层面的优化，不改变模型格式或对外接口，随版本一同发布的还有 macOS Apple Silicon、macOS Intel、iOS XCFramework 以及 Ubuntu x64/arm64/s390x 等多平台预编译包。
通俗解释	MoE 模型可以想象成一家有几百个专科医生的大医院，每来一个病人（token），分诊台要先挑出最合适的几位医生（top-k 专家）再让他们会诊。传统做法是分诊、叫号、会诊分成好几趟跑，中间要反复把名单写下来再读回去。这次改动把 挑专家 这一步在 GPU 上合并成更少的操作，相当于让分诊和叫号在同一个窗口一次办完，减少来回跑腿的开销。专家数量越多（这里是 384 个），这种合并省下的重复劳动就越明显。
主要作用	解决大规模 MoE 模型在 CUDA 上推理时 top-k 专家选择环节的额外开销问题。专家数量达到数百时，路由与选择步骤的调度和内存访问成本会显著上升，把 top-k 融合进更少的算子可以减少中间步骤，从而让这类大 MoE 模型在 NVIDIA GPU 上的本地推理更顺畅。对需要跑大参数量 MoE 开源模型的团队来说，这是直接作用于推理链路的底层改进。
核心功能	一是面向 384 专家规模的 top-k MoE 融合，针对专家数量较多的路由场景做专门处理；二是改动限定在 CUDA 后端，不影响其他后端与模型格式；三是随版本提供 macOS、iOS、Ubuntu 多架构预编译产物，便于直接替换验证；四是提交说明标注由 Codex 辅助完成，体现了 AI 辅助参与底层内核开发的实践。
对比判断	候选摘要未提供该融合前后的吞吐、延迟或显存占用数据，也未与其他后端对比，暂无可信公开对比数据。
适合谁	适合在 NVIDIA GPU 上部署大规模 MoE 开源模型的工程师、做推理内核优化的 CUDA 开发者，以及需要评估不同 llama.cpp 构建版本性能差异的运维与平台团队。

直达链接　https://github.com/ggml-org/llama.cpp/releases/tag/b11953

3. llama.cpp b11962：CUDA 在 MMQ 中融合 MoE gate/up 与 GLU

来源　GitHub Release | 类型　GitHub Release | 主题　推理性能与基础设施　时间　10-28 08:42 | 分数　7

是什么	这是 llama.cpp 的滚动构建 b11962，核心变更是 CUDA 后端在 MMQ（量化矩阵乘）路径中，把 MoE 的 MUL_MAT_ID 门控　上投影（gate/up）与 GLU 激活融合在一起（PR #31188）。它延续了同日 b11953 的 MoE 优化方向，但作用点不同：b11953 处理的是专家选择阶段的 top-k，本版本处理的是专家内部前向计算中的门控投影与 GLU 激活。版本同样附带 macOS、iOS、Ubuntu 多平台预编译包。
通俗解释	MoE 模型里每位被选中的　专家　内部都要做一套固定动作：先把输入分别投影成　门　和　上　两路结果，再用 GLU 这个激活函数把两路合起来，决定哪些信号通过。原来的流程像流水线上的三道独立工序，每道都要把半成品搬进搬出仓库。这次改动把这几道工序合并到同一条流水线上一次完成，中间结果不再反复写回显存。由于它发生在量化矩阵乘（MMQ）路径里，量化模型也能享受到这一合并带来的收益。
主要作用	解决 MoE 模型在 CUDA 量化推理路径中，门控投影与 GLU 激活分步执行带来的额外显存读写与算子调度开销。把 gate/up 与 GLU 融合进 MMQ，可以减少中间张量的落地次数，让量化 MoE 模型的前向计算更紧凑。对于在消费级或单卡 GPU 上跑量化 MoE 模型的场景，这类内核级融合是提升实际可用性的关键手段。
核心功能	一是将 MoE 的 MUL_MAT_ID gate/up 投影与 GLU 激活在 MMQ 中融合，减少中间结果往返；二是作用于 CUDA 量化矩阵乘路径，覆盖量化 MoE 推理场景；三是与同日 b11953 的 top-k 融合形成互补，分别优化专家选择与专家内部计算；四是提供 macOS Apple Silicon、macOS Intel、iOS XCFramework 与 Ubuntu x64/arm64/s390x 预编译包，便于快速验证。
对比判断	候选摘要未给出融合前后的速度、显存或量化精度对比，也未与其他推理框架比较，暂无可信公开对比数据。
适合谁	适合在 GPU 上运行量化 MoE 模型的开发者、做 CUDA 算子融合与量化推理优化的研究者，以及希望跟进 llama.cpp 每日构建以获取最新性能改进的部署团队。

直达链接　https://github.com/ggml-org/llama.cpp/releases/tag/b11962

本期三条均来自 llama.cpp 官方 GitHub Release，其中 b11953 与 b11962 为滚动构建，具体性能收益需以本地实测为准；候选摘要未提供 benchmark 数据，故对比字段统一标注为暂无可信公开对比数据。